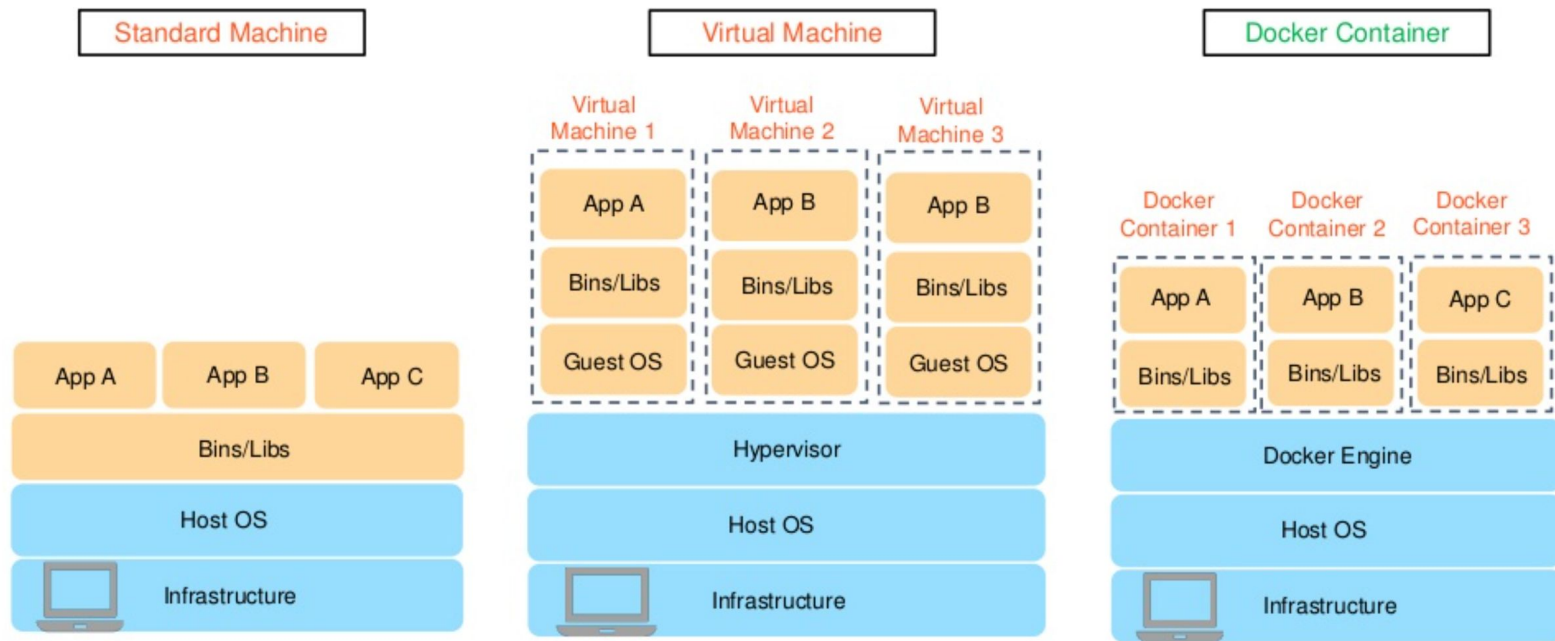


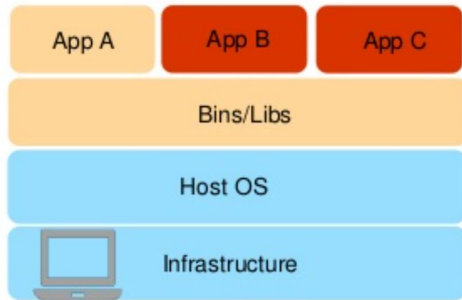
# Контейнери

# Why Docker Container?

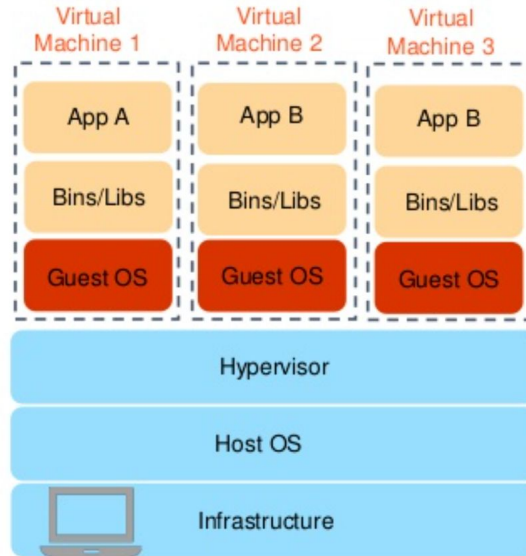


# Why Docker Container?

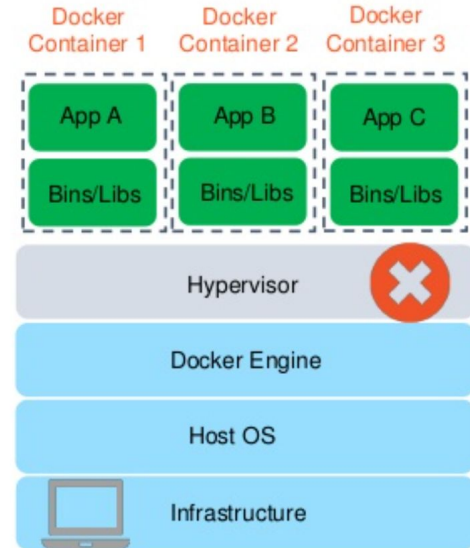
Standard Machine



Virtual Machine

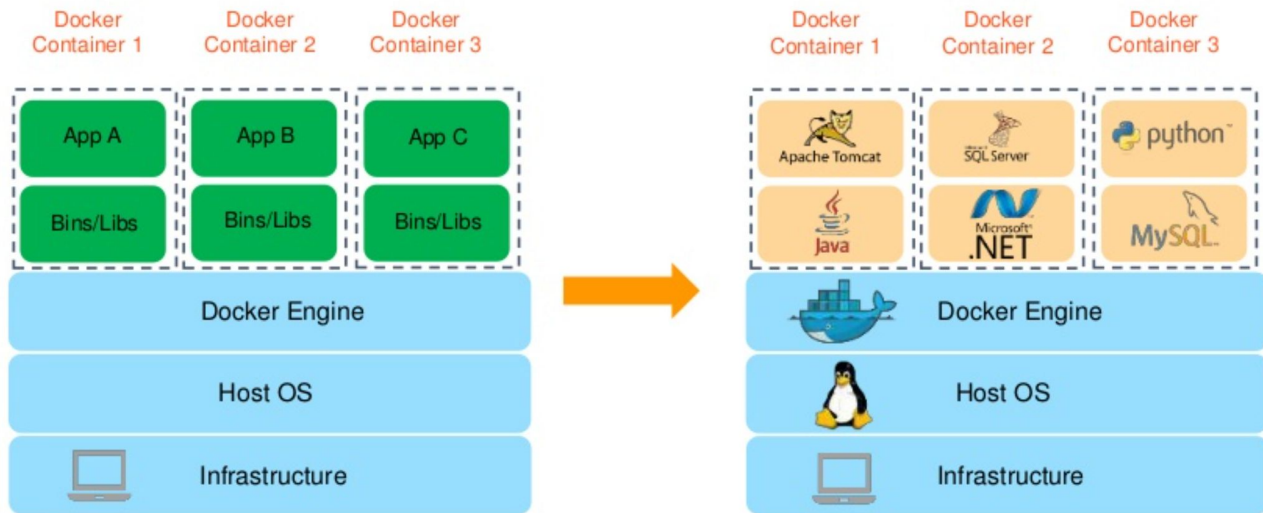


Docker Container

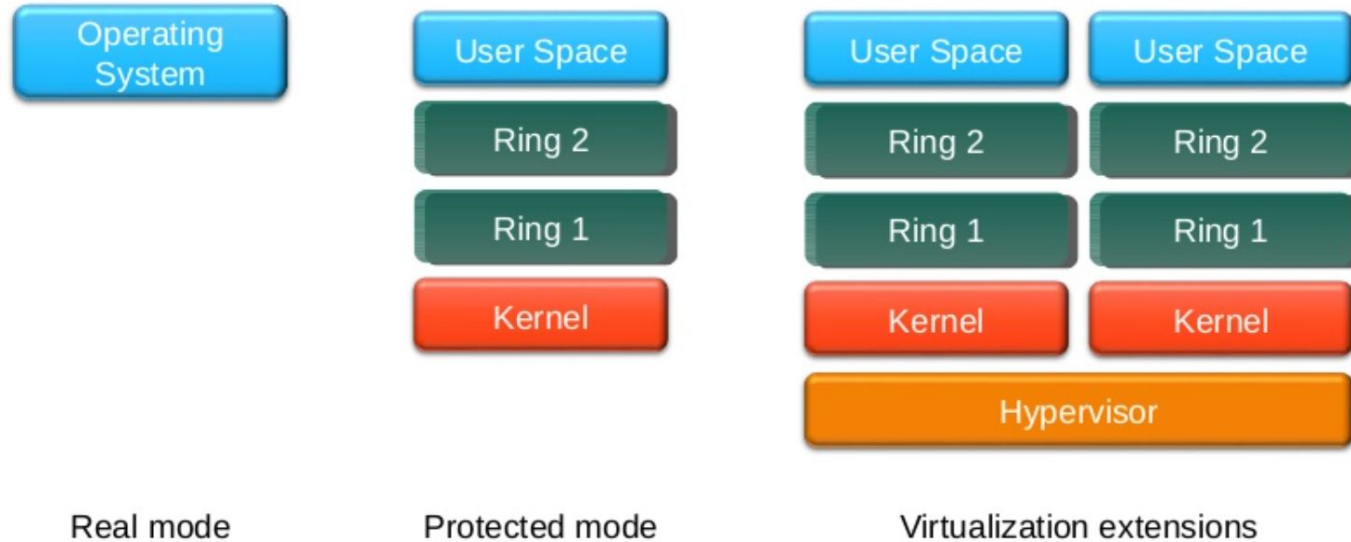


# Why Docker Container?

For example:



# Короткий огляд: віртуальні машини



# Контейнер Vernacular

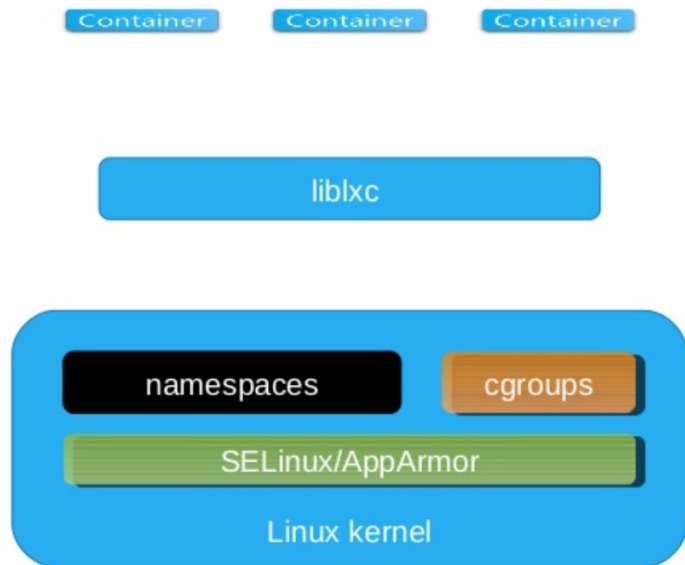
- Контейнер – те, що використовує «Docker» для інкапсуляції служби
  - Не потрібно бути Докером. Альтернативами є CoreOS rkt і Open Container Initiative Microservice
- Сервіс, призначений для виконання одного завдання
  - Подумайте про найменшу одиницю корисної роботи для масштабування Cloud Native Design
- Парадигма, у якій принципи стійкості підприємства не застосовуються (про це пізніше) Pet
- Термін, що стосується тривалої роботи на підприємстві
  - Про домашніх тварин піклуються, і зазвичай їх люблять, плекають і розвивають велику рогату худобу
- Термін, що стосується служби, добробут якої не є пріоритетом
  - Концепція полягає в тому, що велика рогата худоба замінна – вибачте, любителі тварин

# Правила контейнерів – хмарні власні проекти

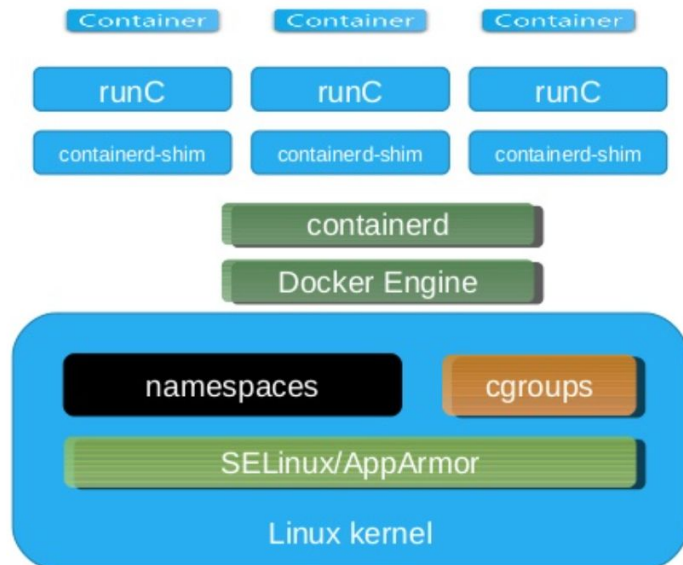
- Контейнери незмінні
  - Створіть один раз, запустіть багато екземплярів
- Контейнери ефемерні
  - Термін служби контейнерів має тривати лише стільки, скільки це абсолютно необхідно
- Контейнерами можна пожертвувати
  - Системи оркестровки можуть припинити роботу контейнера, якщо це необхідно
  - Немає гарантії терміну служби
  - Не зберігайте дані або журнали в контейнерах
- Контейнери обмежують доступ до ресурсів
  - Визначте контрольні групи для доступу до CPU/RAM
  - Уникайте використання облікових даних ROOT
  - Багаторівнева файлова система допомагає керувати сховищем

# Що таке контейнер?

## Linux Containers



## Docker 1.10 and later





# Концепції контейнерів

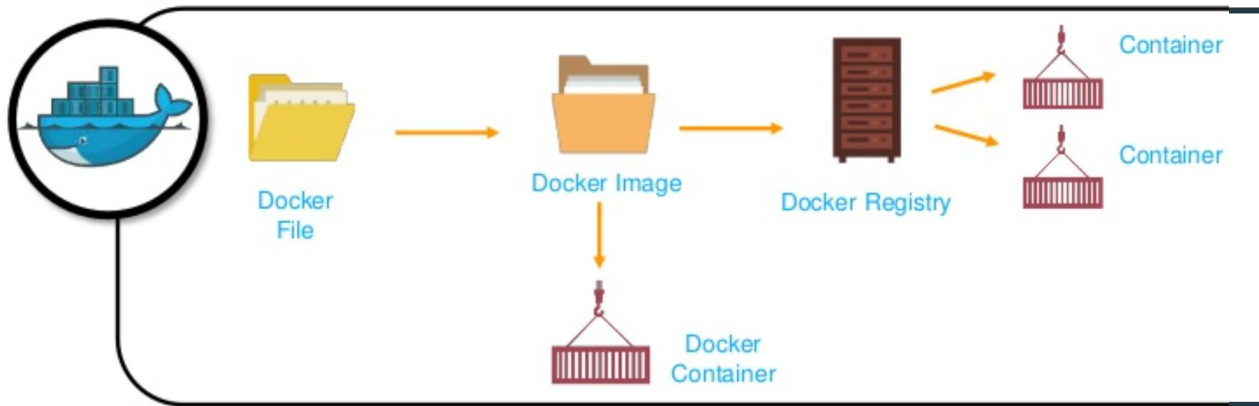
- Подання заявки
- Створено з «Dockerfile»
- Використовує багат шарову файлову систему
- Один процес для однієї програми
- Створено в двигуні Docker
  - Відображається як локальне двійкове сховище
- Зберігається в бінарному сховищі, відомому як образ контейнера реєстру

# Реєстр контейнерів

- Де зображення контейнерів зберігаються поза локальним механізмом
- Основні команди
  - `тег docker` розміщує змінний тег на зображенні
  - `docker push` завантажує зображення до реєстру
  - `docker pull` завантажує образ із реєстру
- `docker.io` за замовчуванням (він же Docker Hub «попередньо визначений» як надійний
  - Будьте чіткі щодо джерела зображення та підтвердіть його

# How to create a Docker Container?

- There are multiple Docker images available in the registry and all can be retrieved through the Docker pull command (e.g. Docker pull image\_name)
- Once a Docker Image is retrieved from the Docker Registry, a user can get the Docker Image and build new Containers

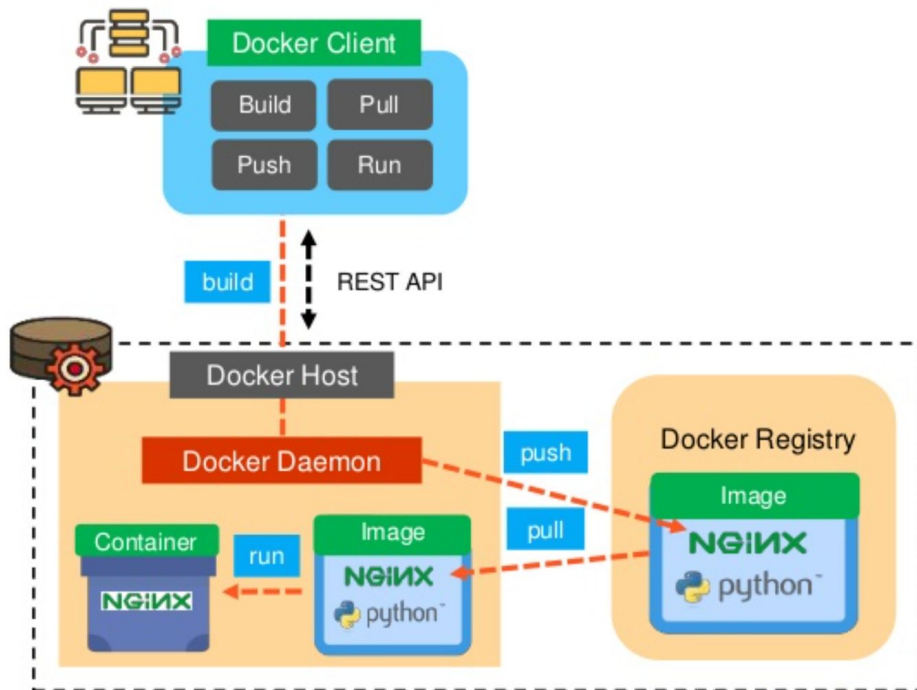


# How to create a Docker Container?

In Docker, a Dockerfile is used to build the image using *build command* and that image is stored into the registry using *push command*

When you run *pull command*, Docker Image (NGINX) is retrieved from the registry

Finally, a single Container (NGINX) is built using Docker Image through the *run command*



# Як створити контейнер Docker?

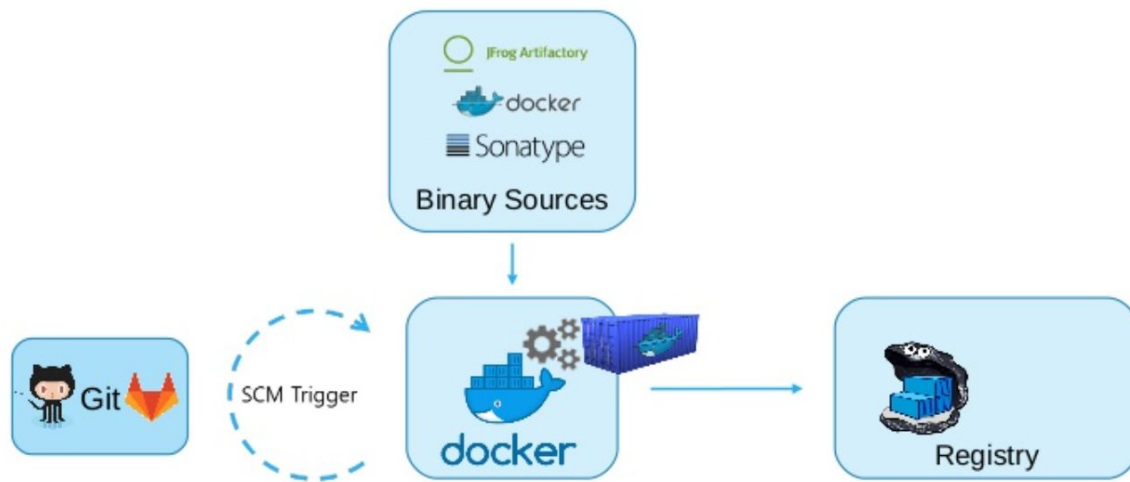
## **ЧИ ЗНАЛИ ВИ?**

- Коли створюється контейнер, на верхівці шарів образу Docker формується новий шар, який називається **шар контейнера**.
- Кожен контейнер має окремий (**читання/запис**) шар контейнера, і всі зміни, зроблені в Docker-контейнері, відображаються лише в цьому конкретному шарі.
- Якщо контейнер видаляється, то разом із ним видаляється і шар контейнера.

## **Примітка:**

Образ Docker має кілька шарів, і кожен шар образу створюється під час виконання кожної команди у Dockerfile.

# Процес створення зображення контейнера



# Як створюються зображення – основні компоненти Dockerfile

```
FROM centos:centos7
MAINTAINER Black Duck Hub Team
RUN yum -y update-minimal --security --sec-severity=Important --sec-severity=Critical --setopt=tsflags=nodocs && yum
clean all && chmod -x /bin/sh
```

```
ARG bds_ver
ARG LASTCOMMIT
ARG BUILDTIME
ARG BUILD
ENV APP_HOME=/scan.cli-${bds_ver}
ENV PATH=${APP_HOME}/bin:${JAVA_HOME}/bin:${PATH}
```

```
COPY ./output/ose_scanner /ose_scanner
COPY ./hub_scanner/scan.cli/scan.cli-${bds_ver} /scan.cli-${bds_ver}
COPY ./LICENSE /LICENSE
COPY ./LICENSE /licenses/
COPY ./help.1 /help.1
```

```
LABEL name="Black Duck OpsSight for OpenShift Scanner"
ENTRYPOINT [ "/ose_scanner" ]
EXPOSE 9036
```

# Як створюються зображення – кеш Docker Engine

```
[root@localhost ose-scanner]# docker images
```

| REPOSITORY         | TAG     | IMAGE ID     | CREATED      | SIZE     |
|--------------------|---------|--------------|--------------|----------|
| hub_ose_controller | 4.2.0   | 363bfa6e7974 | 15 hours ago | 53.98 MB |
| hub_ose_arbiter    | 4.2.0   | 15adc3ecc6ab | 16 hours ago | 53.38 MB |
| hub_ose_scanner    | 4.2.0   | 395dfd09d6d7 | 16 hours ago | 447 MB   |
| docker.io/centos   | centos7 | 196e0ce0c9fb | 6 weeks ago  | 196.6 MB |



# Як створюються зображення – історія шару зображення

```
[root@localhost ose-scanner]# docker history hub_ose_scanner:4.2.0
```

| IMAGE        | CREATED      | CREATED BY                                    | SIZE     |
|--------------|--------------|-----------------------------------------------|----------|
| 395dfd09d6d7 | 16 hours ago | /bin/sh -c #(nop) EXPOSE 9036/tcp             | 0 B      |
| 6ccb88892a15 | 16 hours ago | /bin/sh -c #(nop) ENTRYPOINT ["/ose_scanner"] | 0 B      |
| 604fcdb13b57 | 16 hours ago | /bin/sh -c #(nop) LABEL name=Black Duck OpsS  | 0 B      |
| 5bf60b767227 | 2 days ago   | /bin/sh -c #(nop) COPY file:0319ebe1148b5cefa | 682 B    |
| fe23aeab7fcc | 2 days ago   | /bin/sh -c #(nop) COPY file:e822182ba43798ba0 | 11.36 kB |
| 9cdc179735ad | 2 days ago   | /bin/sh -c #(nop) COPY file:e822182ba43798ba0 | 11.36 kB |
| 57bd5e62be14 | 2 days ago   | /bin/sh -c #(nop) COPY dir:d9dc3b531575096c83 | 241.6 MB |
| a1cb8fd37a68 | 2 days ago   | /bin/sh -c #(nop) COPY file:98c69c969ee05b51b | 6.14 MB  |
| 13855a218a3e | 7 days ago   | /bin/sh -c #(nop) ENV PATH=/scan.cli-4.2.0/b  | 0 B      |
| 885efab8f9b5 | 7 days ago   | /bin/sh -c #(nop) ENV APP_HOME=/scan.cli-4.2  | 0 B      |
| 1ed791e999b5 | 7 days ago   | /bin/sh -c #(nop) ARG BUILD                   | 0 B      |
| 9dc95a5ceb4  | 7 days ago   | /bin/sh -c #(nop) ARG BUILDTIME               | 0 B      |
| 8ada27a4da06 | 7 days ago   | /bin/sh -c #(nop) ARG LASTCOMMIT              | 0 B      |
| 7461b836791f | 7 days ago   | /bin/sh -c #(nop) ARG bds_ver                 | 0 B      |
| 4020be54fb0f | 7 days ago   | /bin/sh -c yum -y update-minimal --security - | 2.632 MB |
| 208a012b6fe4 | 7 days ago   | /bin/sh -c #(nop) MAINTAINER Black Duck Hub   | 0 B      |
| 196e0ce0c9fb | 6 weeks ago  | /bin/sh -c #(nop) CMD ["/bin/bash"]           | 0 B      |
| <missing>    | 6 weeks ago  | /bin/sh -c #(nop) LABEL name=CentOS Base Ima  | 0 B      |
| <missing>    | 6 weeks ago  | /bin/sh -c #(nop) ADD file:1ed4d1a29d09a636dd | 196.6 MB |

# Image Layer Cache Impacts Build

```
[root@localhost ose-scanner]# docker images --all
```

| IMAGE        | REPOSITORY       | TAG     | IMAGE ID     | CREATED      | SIZE     |
|--------------|------------------|---------|--------------|--------------|----------|
| 395dfd09d6d7 | hub_ose_scanner  | 4.2.0   | 395dfd09d6d7 | 17 hours ago | 447 MB   |
| 6ccb88892a15 | <none>           | <none>  | 6ccb88892a15 | 17 hours ago | 447 MB   |
| 604fcdb13b57 | <none>           | <none>  | 604fcdb13b57 | 17 hours ago | 447 MB   |
| 5bf60b767227 | <none>           | <none>  | 5bf60b767227 | 2 days ago   | 447 MB   |
| fe23aeab7fcc | <none>           | <none>  | fe23aeab7fcc | 2 days ago   | 447 MB   |
| 9cdc179735ad | <none>           | <none>  | 9cdc179735ad | 2 days ago   | 447 MB   |
| 57bd5e62be14 | <none>           | <none>  | 57bd5e62be14 | 2 days ago   | 447 MB   |
| a1cb8fd37a68 | <none>           | <none>  | a1cb8fd37a68 | 2 days ago   | 205.4 MB |
| 13855a218a3e | <none>           | <none>  | 13855a218a3e | 7 days ago   | 199.3 MB |
| 885efab8f9b5 | <none>           | <none>  | 885efab8f9b5 | 7 days ago   | 199.3 MB |
| 1ed791e999b5 | <none>           | <none>  | 1ed791e999b5 | 7 days ago   | 199.3 MB |
| 9dcb95a5ceb4 | <none>           | <none>  | 9dcb95a5ceb4 | 7 days ago   | 199.3 MB |
| 8ada27a4da06 | <none>           | <none>  | 8ada27a4da06 | 7 days ago   | 199.3 MB |
| 7461b836791f | <none>           | <none>  | 7461b836791f | 7 days ago   | 199.3 MB |
| 4020be54fb0f | <none>           | <none>  | 4020be54fb0f | 7 days ago   | 199.3 MB |
| 208a012b6fe4 | <none>           | <none>  | 208a012b6fe4 | 7 days ago   | 196.6 MB |
| 196e0ce0c9fb | docker.io/centos | centos7 | 196e0ce0c9fb | 6 weeks ago  | 196.6 MB |
| <missing>    |                  |         |              |              |          |
| <missing>    |                  |         |              |              |          |

# Проблеми довіри до контейнера

- Звідки насправді береться ваше базове зображення?
- Яка справність цього базового зображення?
- Ви оновлюєте його під час збирання, але з якого кешу?
- Ви довіряєте своїм серверам збірки, але хто їх контролює?
- Хто має право змінювати зображення контейнерів?
- Що станеться, якщо базовий реєстр зображень зникне?
- Що станеться, якщо тег базового зображення зникне?
- Що станеться, якщо дзеркало оновлення вийде з ладу?
- Як виправити зображення контейнерів?

# Справність зображення має вирішальне значення для безпеки зображення

## Docker Hub Container Scanning

OFFICIAL REPOSITORY

centos ☆

Last pushed: 2 months ago

Repo info Tags

### Scanned Images

|                |                        |                    |                                |
|----------------|------------------------|--------------------|--------------------------------|
| 7.4.1708       | Compressed size: 73 MB | Scanned 3 days ago | This image has vulnerabilities |
| centos7.4.1708 | Compressed size: 73 MB | Scanned 3 days ago | This image has vulnerabilities |
| 7              | Compressed size: 73 MB | Scanned 3 days ago | This image has vulnerabilities |
| centos7        | Compressed size: 73 MB | Scanned 3 days ago | This image has vulnerabilities |

## Red Hat Container Catalog Health Index

Red Hat Enterprise Linux 7 ☆

by Red Hat, Inc. | in Product Red Hat Enterprise Linux

registry.access.redhat.com/rhel17/rhel1 Updated 12 days ago 7.4-120 Health Index A

Overview

Get this image

Tech Details

Documentation

Tags



# Керування томами Docker

Мета: обмін даними між контейнером і зовнішнім світом

Основні команди

- `docker volume create` - створити том
- `docker run image --to source:/data map source to /data` у контейнері

Підтримувані драйвери

- `local`, `tmpfs`, `btrfs`, `nfs` представляють змонтовані файлові системи на хості

Використовуйте в Dockerfile

## Modifies build image - Wrong

```
FROM centos:centos7
VOLUME /data
RUN touch /data/x
```

## Modifies data in built image - Correct

```
FROM centos:centos7
RUN mkdir /data && touch /data/x
VOLUME /data
```

# Управління секретами в контейнерах

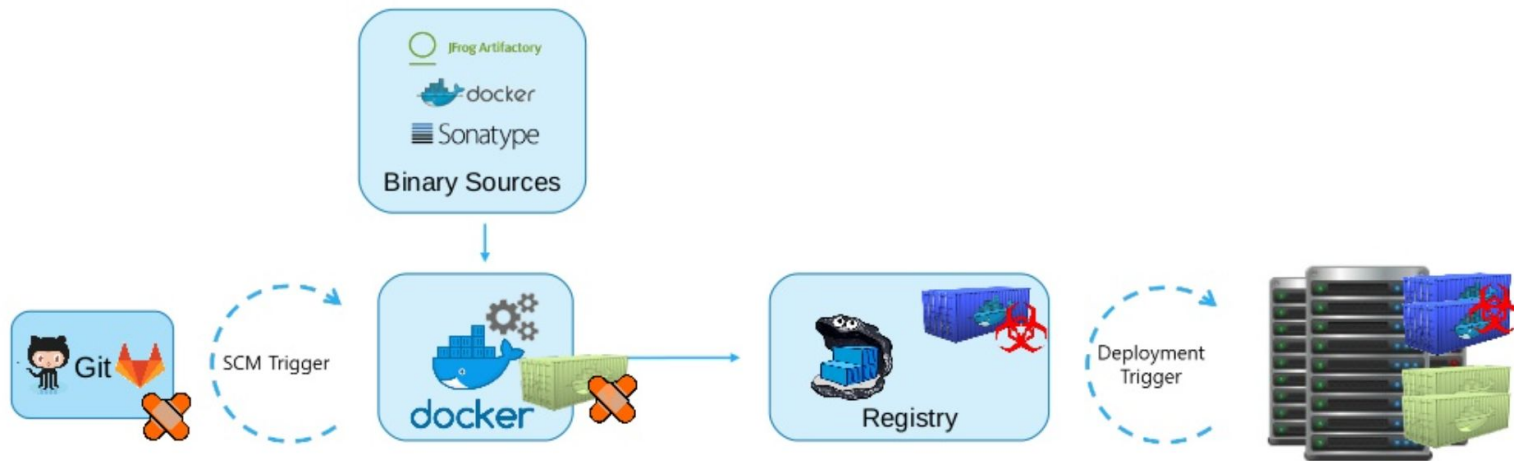
Секретами є...

- Ключі шифрування програми
- Маркери доступу

Секретами облікових даних керує...

- Змінні середовища – невеликі установки та сценарії розробників
- JSON Web Tokens (JWT)
- Docker Compose у режимі Swarm (Compose V3)
- Зовнішні секретні служби (HashiCorp Vault, Keywhiz, Consul, Amazon KMS тощо)

# Виправлення контейнерів – А/В тестування за допомогою Rebuild Git



# Налагодження контейнера

## **`docker run myimage -ti`**

- Запускає новий інтерактивний екземпляр зображення контейнера ``myimage`` із точкою входу за замовчуванням

## **`docker run myimage -ti /usr/bin/sh`**

- Запускає новий інтерактивний екземпляр зображення контейнера ``myimage``, відкриваючи оболонку

## **`docker attach name`**

- Приєднується до запущеного екземпляра контейнера під назвою ``name``

## **`docker exec name -ti /usr/bin/sh`**

- Приєднується до запущеного екземпляра контейнера під назвою ``name`` і відкриває оболонку

## **`docker logs myimage --follow`**

- Приєднується до запущеного екземпляра контейнера під назвою ``name``, що відкривається та слідує стандартному виводу



# Базові команди Docker Container

## Basic commands for Docker

- **Docker Container commit** - Command to create a new Docker image from the changes made in Container
- **Docker Container cp** - Command to copy files between the local filesystem and a Docker Container
- **Docker Container prune** - Command to remove all stopped Containers
- **Docker Container kill** - Command to terminate one or more running Containers
- **Docker Container exec** - Command to run a new command in a running Container
- **Docker Container ls** - Command to list Docker Containers
- **Docker Container rm** - Command to remove one or more Containers
- **Docker Container restart** - Command to restart one or more Containers

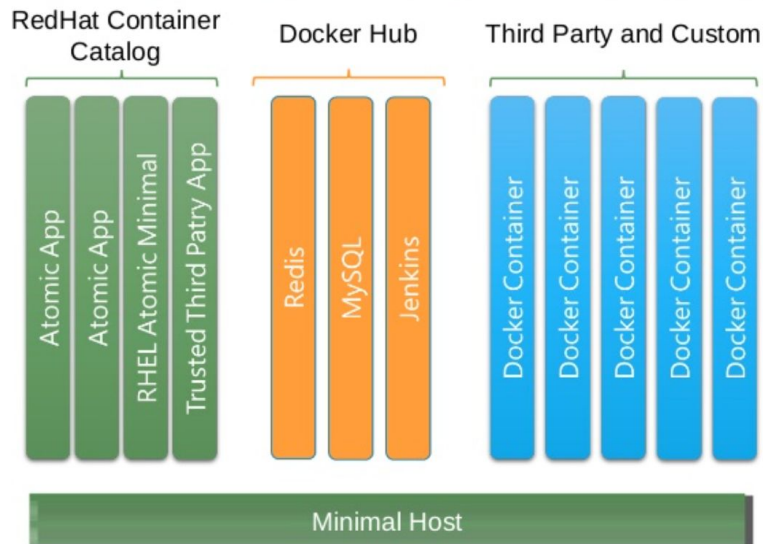


# Захист вмісту та середовища контейнера

# Вимоги до розгортання

Проблема: Кому довіряти і чому?

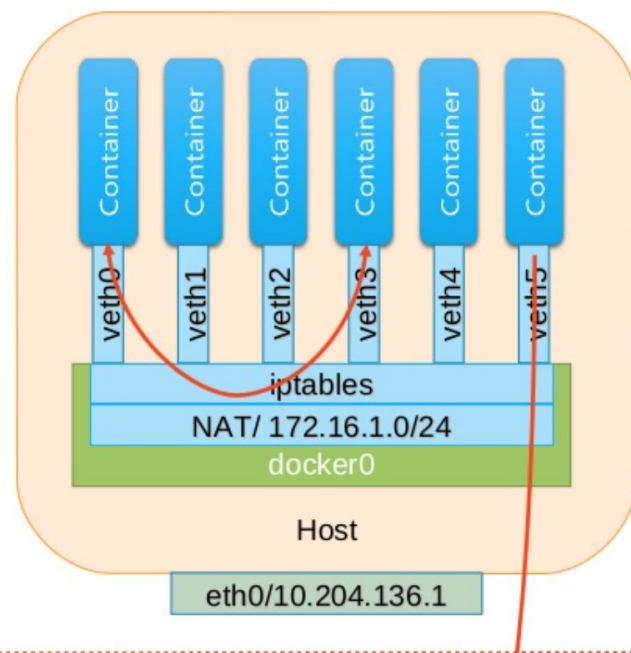
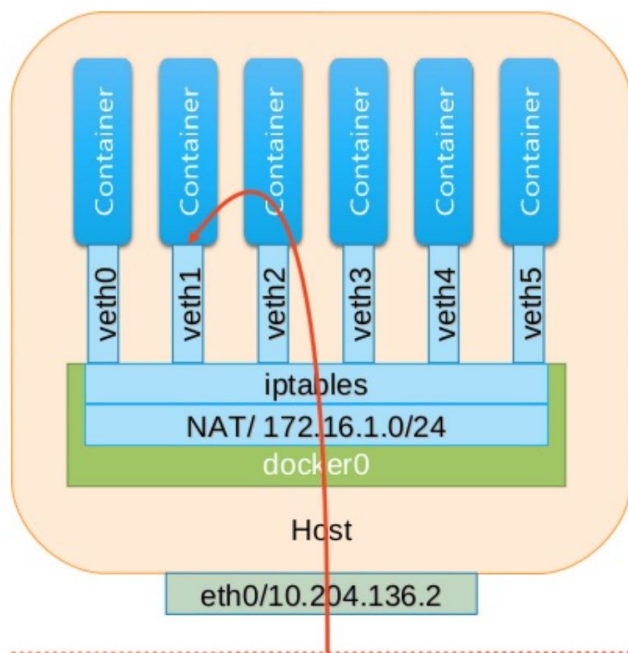
- Надійне джерело?
- Неочікуваний вміст зображення
- Заблоковані версії прикладного рівня (наприклад, без оновлення yum)
- Залежності рівня (монолітні та мікросервіси)
- Коли перевірено?
- Надійна платформа
- Підписано ким?



# Розгортання – визначте розумні мережеві політики

- Стандартною мережею Docker є Linux Bridge
- Політика доступу визначена в iptables
  - На основі запуску демона Docker
- Зовнішній зв'язок увімкнено за замовчуванням
  - iptables=off, щоб вимкнути модифікацію iptables
- Зв'язок між контейнерами ввімкнено за замовчуванням
  - icc=false, щоб вимкнути зв'язок між контейнерами
  - link=CONTAINER\_NAME\_or\_ID:ALIAS з портами EXPOSE з файлу Docker
  - Усі зв'язки між контейнерами/хостами є зовнішніми
- Команда ``docker network`` спрощує аспекти проектування мережі
  - Створюйте визначені користувачем мережі, включаючи накладені мережі
  - `docker network create --driver bridge sql`

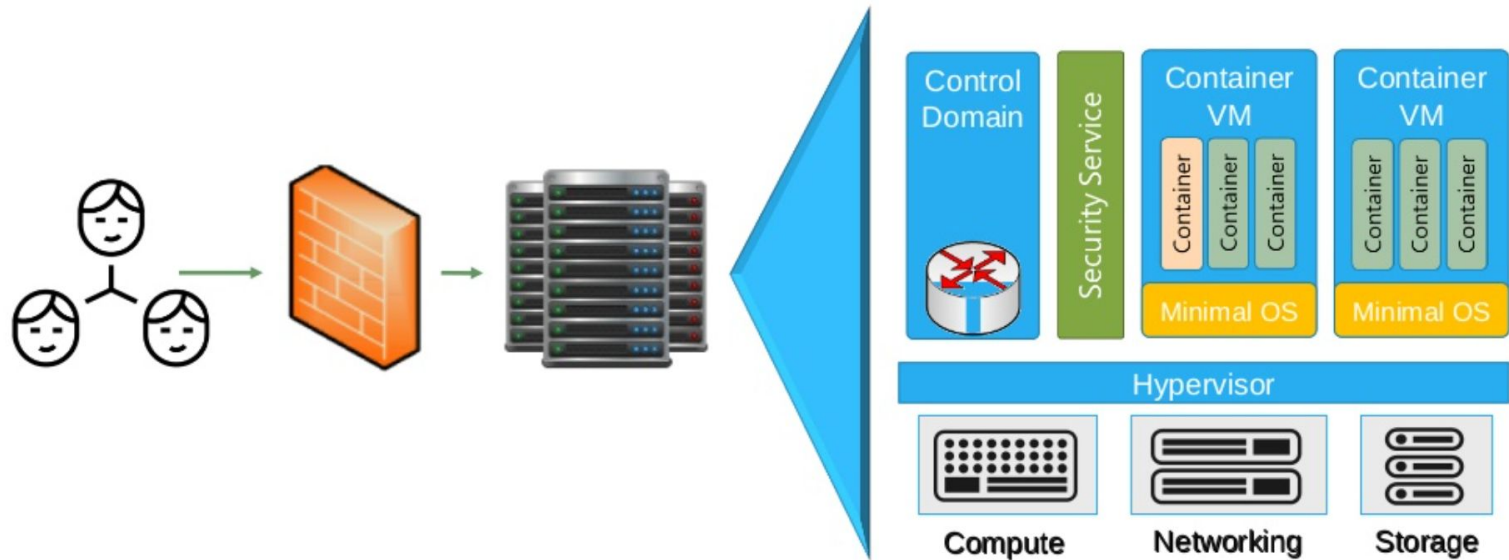
# Мережа Docker - приклад



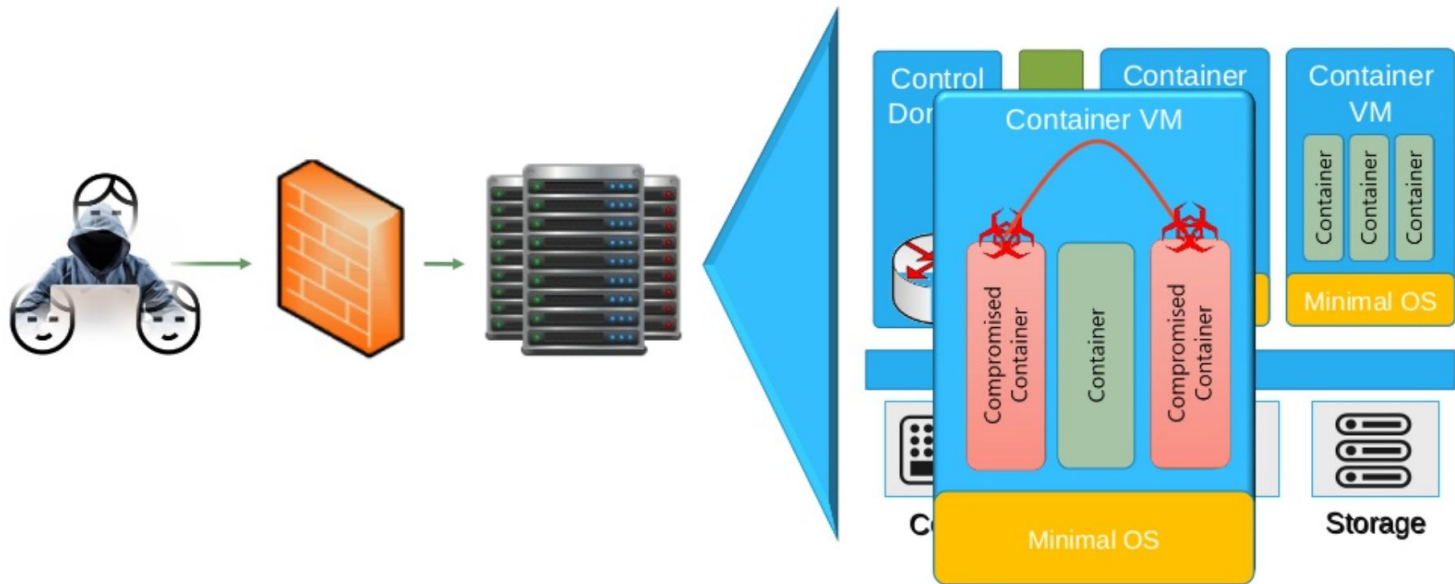
# Захистіть хост і простори імен

- Увімкніть модулі безпеки Linux
  - SELinux
    - з підтримкою selinux на механізмі Docker, `--security-opt="label:profile"`
  - AppArmor
    - `security-opt="apparmor:profile"`
- Застосуйте профілі безпеки ядра Linux
  - grsecurity, захист PaX і seccomp для ALSR і RBAC
- Налаштуйте привілейовані можливості ядра
  - Зменшіть можливості за допомогою `--cap-drop`
  - Остерігайтеся `--cap-add` і `--privileged=false` і `CAP_SYS_ADMIN`
- Використовуйте мінімальний хост-ОС Linux
  - Red Hat Linux Atomic Host, Container Linux, RancherOS
- Зменшити вплив галасливих сусідів
  - Використовуйте контрольні групи, щоб установити спільні ресурси процесора та пам'ять

# Рівень безпеки контейнера для захисту від атак



# Рівень безпеки контейнера для захисту від атак





# Питання та відповіді